

P51: Apprendista

Business Process Modelling Course Project

Students

Marco Tamberi — 621490

Fabrizio Anelli — 545819

Academic Year 2025/2026

Chapter 1

Introduction

1.1 Scenario

Let's consider the scenario in which an apprentice is serving an apprenticeship in a workshop.

The process begins with a brief interview between the owner and the apprentice. At the end of the interview, the owner assigns a supervisor to the apprentice. The latter then contacts his supervisor to ask what activities to perform. Iteratively, the supervisor suggests an activity and the apprentice asks instruction on how to perform it, receives them, and tries to execute it. If he isn't successful, the supervisor can ask him to repeat it instead of assigning him a new one, and the apprentice can accept or declare that he is not able to perform that activity. In the latter case, the supervisor can insist or assign him a new activity, and so on.

At the end of the apprenticeship period, the supervisor sends a report to the owner. After the evaluation of the report, the owner can decide to offer a contract to the apprentice, to extend the period of apprenticeship, or to end the relationship.

The aim of the project is to model appropriate processes that accurately mirror the scenario described above and that interact correctly.

1.2 Tools

The BPMN model has been built and validated using **Camunda Modeler**. The Workflow Nets have been built, designed and analyzed using **WoPeD**, which offers in a single interface the statistics of the net, the verification of the properties of the net, the analysis of soundness (boundedness and liveness), and the coverability graph.

Chapter 2

BPMN Model

2.1 Design Choices

The scenario under analysis involves 2 autonomous actors, modeled as two **pools**: the *Apprentice* and the *Workshop*. Inside the workshop two roles are acting, the *Owner* and the *Supervisor*, modeled as two **lanes** in the same pool: they are part of the same organization and exchange work internally, so they are connected with **sequence flows** instead of messages. The communication between the two pools happens via **messages flows**.

The main design choices were:

- **Only one start and one end for each pool.** This choice ensures that each orchestration can be transformed into a workflow net. The Apprentice does not initiate the process autonomously: the process starts when he receives the call-up message from the Owner. Therefore, the Apprentice pool begins with a message start event.
- **Event-Based Gateways for deferred choices.** There are two instances in which the actor doesn't choose based on his own will, but has to wait and continues based on *which message arrives*, a choice determined by the other actor (deferred choice). These are the two "hubs" *Attesa* (on the apprentice side) and *Esito* (on the supervisor

side). A data-based XOR wouldn't be accurate, because it would decide based on an information not yet locally available and could cause a deadlock; on the other hand, the event-based gateway waits and triggers as soon as the first message arrives.

- **No implicit gateways.** The split and the join of the control flow happen only through explicit gateways. Consecutive activities without bifurcations don't require gateways.

2.2 The Diagrams

First, we present the two pools taken individually, then we will show the complete collaboration, in which the pools are connected by messages flows.

2.2.1 Apprentice Pool

He is a reactive participant: he is called up by a message from the owner, gets interviewed and, after a supervisor is assigned, contacts him. From here, he waits (event-based hub *Attesa*) and, based on the message he receives, performs an activity (asks for instructions, tries, communicates the outcome), handles an eventual request for repeating the task, which he can accept or decline based on the feasibility of the task, or receive the final decision. Each branch of the cycle brings him back to waiting, and the flow ends with apprentice receiving a contract proposal, an extension of the internship, or the conclusion of the relationship with the workshop.

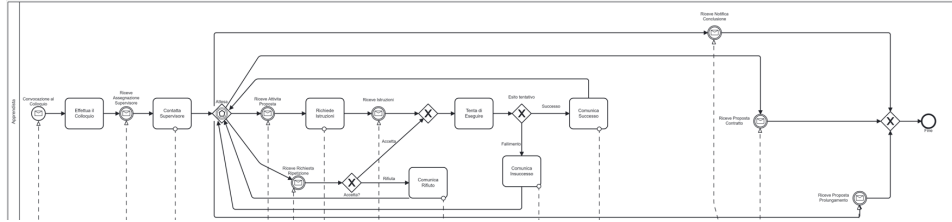


Figure 2.1: Apprentice pool.

2.2.2 Workshop Pool

It involves two roles in as many lanes, connected by sequence flows. The *Owner* plans and conducts the interview, assigns the supervisor, and, at the end of the internship period, evaluates the report and decides the outcome (contract proposal, apprenticeship extension proposal, end of relationship). The *Supervisor* leads the work cycle: he proposes an activity, gives the apprentice instructions, and, via the event-based hub *Esito*, reacts to the success, failure or refusal to repeat the task by the apprentice (eventually asking him to repeat it), until he needs to prepare the report and send it to the owner.

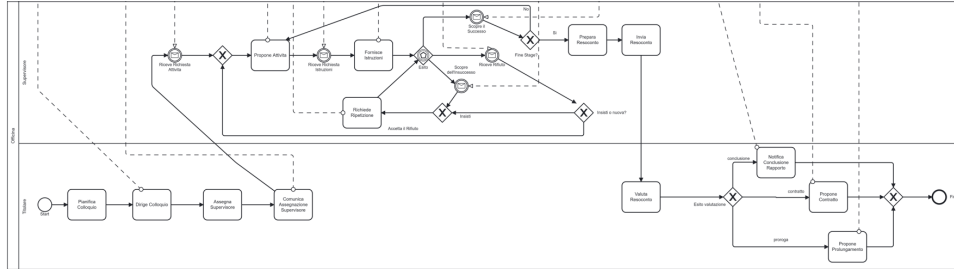


Figure 2.2: Workshop Pool (Owner and Supervisor lanes).

2.2.3 Complete Collaboration

Puts in communication the two pools through 13 message flows: call up and assignment of the supervisor, the request and proposal of the activities, exchange of instructions and outcomes, handling of repetitions and refusals, and finally the communications of the final decision of the owner.

It is here that the deferred choices of the two event-based hubs are translated into effective synchronizations among participants.

Chapter 3

From BPMN to Workflow Nets

3.1 Transformation Rules

For each orchestration and for the collaboration, the same transformation is applied:

- A starting place i and a ending place o are added;
- **Activities** and **Events** become **Transitions**;
- **Sequence Flows** become **Places**;
- The **XOR** and **Event-Based Gateways** become **Places**: a choice is a place with multiple outgoing transitions, a join is a place with multiple incoming transitions;
- A branch that connect two gateways directly (that is, place-place) requires a **Silent Transition** (*dummy*): in a Petri Net two places can't be adjacent, and that branch represents only a routing choice, without any observable activity. These are the **No** branch of *Fine Stage?*, the **Insisti** and **Accetta il rifiuto** branches, and the **Accetta** branch of the *Accetta?* gateway.

3.2 Collaboration Net

The complete net is the union of the two orchestration nets, to which **a place for each message flow** is added (13 in total): the token in said places represents a message in transit. Consequently, each *sending* becomes a **fork** (the actor goes forward *and* sends) and each **reception** a **join** (the actor restarts only when the message arrives). Since the apprentice is triggered by the call-up message, the only start of the collaboration is the workshop's *Start*; to obtain a single end an auxiliary transition, that connects the two endings in the place *o* is added.

Chapter 4

Nets Analysis

For each nets, the following properties were checked: workflow nets properties, soundness (option to complete + proper termination + absence of dead transitions), safeness (1-boundedness), free-choice, well-handledness, coverability with S-components, and the dimension of the coverability graph. Table 4.1 summarizes the statistics, while table 4.2 the properties.

Table 4.1: Statistics of the nets and dimension of the coverability graph (CG).

Net	Places	Transitions	Arcs	Vertices CG	Arcs CG
Apprentice	12	17	34	12	17
Workshop	18	23	46	18	23
Collaboration	43	41	108	44	57

4.1 Orchestration: Apprentice

The apprentice's net is a **sound** and **safe** workflow net. Each transition has exactly one incoming and one outgoing place: it is a **Finite State Machine (S-net)**, so only one token is in circulation, and is **free-choice**, **well-handled** and **well-structured**, **S-coverable** by only one S-component.

Table 4.2: Properties of the three nets.

Property	Apprentice	Workshop	Collaboration
Workflow Net	Yes	Yes	Yes
Soundness	Yes	Yes	Yes
Safeness	Yes	Yes	Yes
Free-choice	Yes	Yes	No (2 violations)
Well-handledness	Yes	Yes	No (handle PT/TP)
S-coverability	Yes (1)	Yes (1)	No (concurrency)

The coverability graph has 12 vertices and 17 arcs, and is limited.

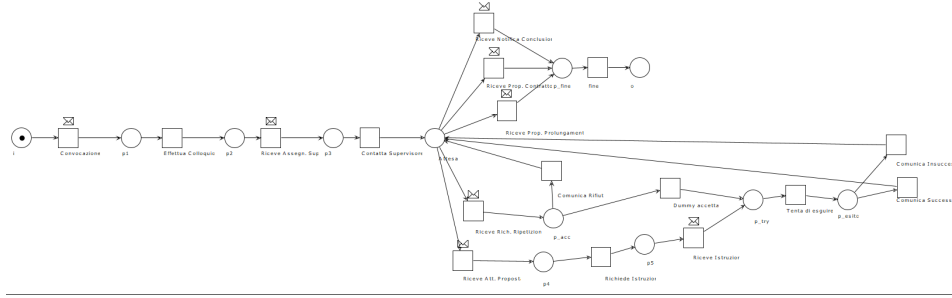


Figure 4.1: Apprentice's workflow net. The transitions with the envelope are points of communication; internal transitions (e.g. *Tenta di eseguire*, *Dummy accetta*) don't exchange messages.

4.2 Orchestration: Workshop

The workshop's (owner + supervisor) workflow net is, too, a **sound** and **safe** workflow net, a **Finite State Machine (S-net)**, so it is **free-choice**, **well-handled**, **well-structured** and **S-coverable** by a single S-component. The choices (the event-based gateway *Esito*, and the gateways *Fine Stage?*, *Insisti o nuova?* and *Esito valutazione*) are all free-choice because each choice place is the only incoming place of the transitions that it feeds into. The

coverability graph has 18 vertices and 23 arcs.

4.3 Complete Collaboration

The complete net is a **safe** and **sound** workflow net, as checked on the reachability graph: all the 44 reachable markings are 1-bounded, from every marking the final marking can be reached, and there are no dead transitions. Unlike the orchestrations, however, it is **not free-choice nor well-handled**.

- **Free-Choice Violations (2).** They are exactly in the two event-based gateways: *Attesa* (apprentice side) and *Esito* (supervisor). The place-hub feeds into multiple receiving transitions, but each of them has a second different input (its respective place-message): which branch is triggered is determined by the incoming message, so the other participant, so the choice is not free.
- **Handle PT/TP.** The forks on the sending and the joins on reception create synchronization paths between the two pools, from which the handles that violate the well-handledness.
- **S-Components and Safeness.** The presence of concurrency (the two pools proceed in parallel, synchronized by the messages) prevent the net from being a finite state machine: the two pools constitute two S-components, but the place-messages are not covered by any S-component. Consequently, the net is not S-covered and the safeness can't be deduced from the S-invariants. In fact, it has been checked *behaviourally* based on the reachability graph.

This is the focal point of the composition: *the soundness of the parts does not guarantee the soundness of the whole*, thus it has to be checked separately. In our case, the whole remains safe and sound, and only loses the structural properties of free-choice and well-handledness, an inevitable con-

sequence of the asynchronous communication/deferred choice among participants, and not a fault of the model.

The coverability graph has 44 vertices and 57 arcs.

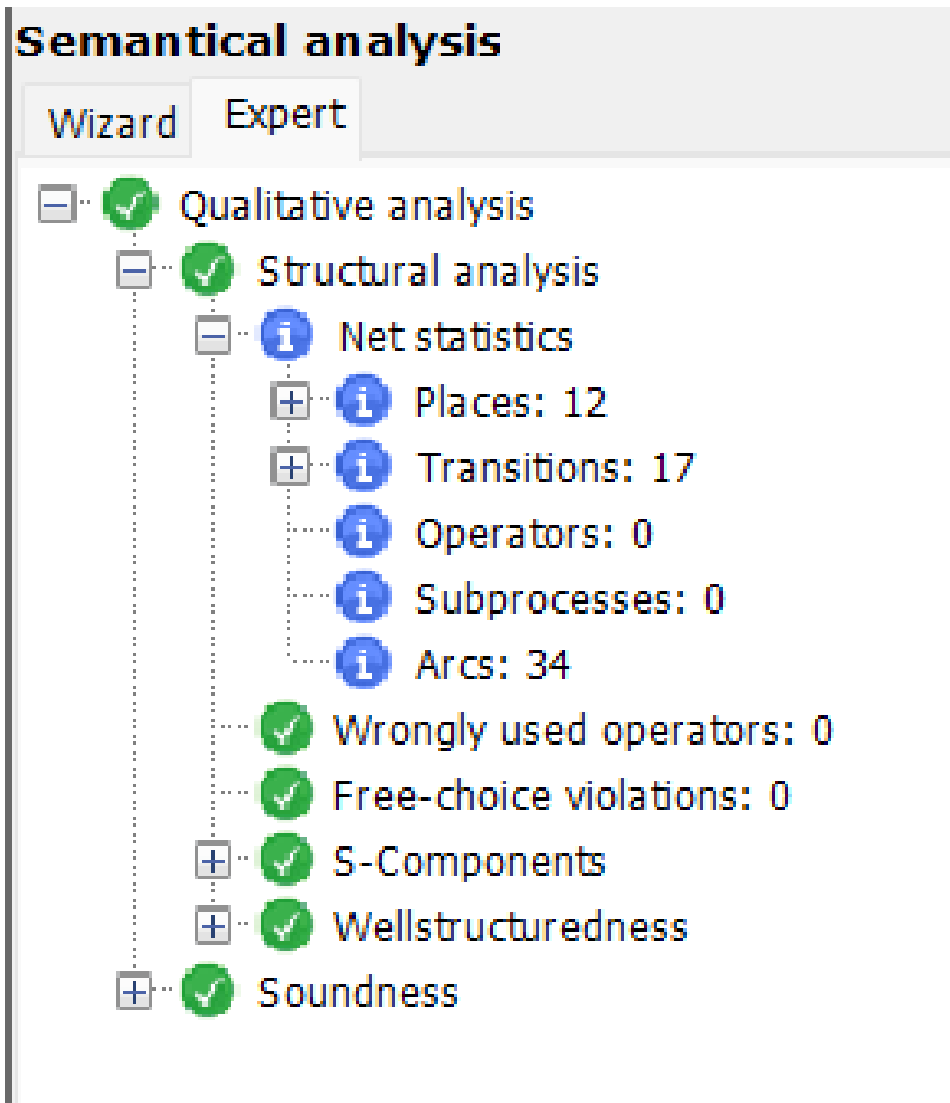


Figure 4.2: Semantic analysis in WoPeD: verified workflow net property and soundness; 12 places, 17 transitions, 34 arcs.

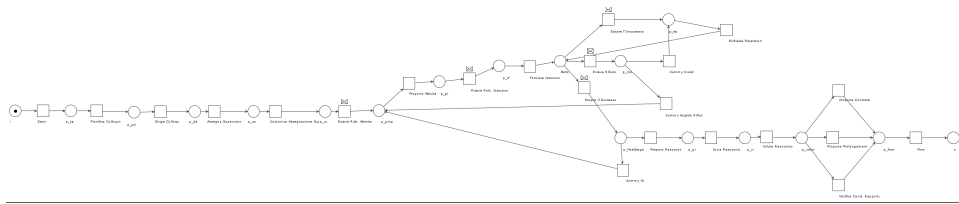


Figure 4.3: Workshop’s workflow net (Owner and Supervisor lanes in a single orchestration).

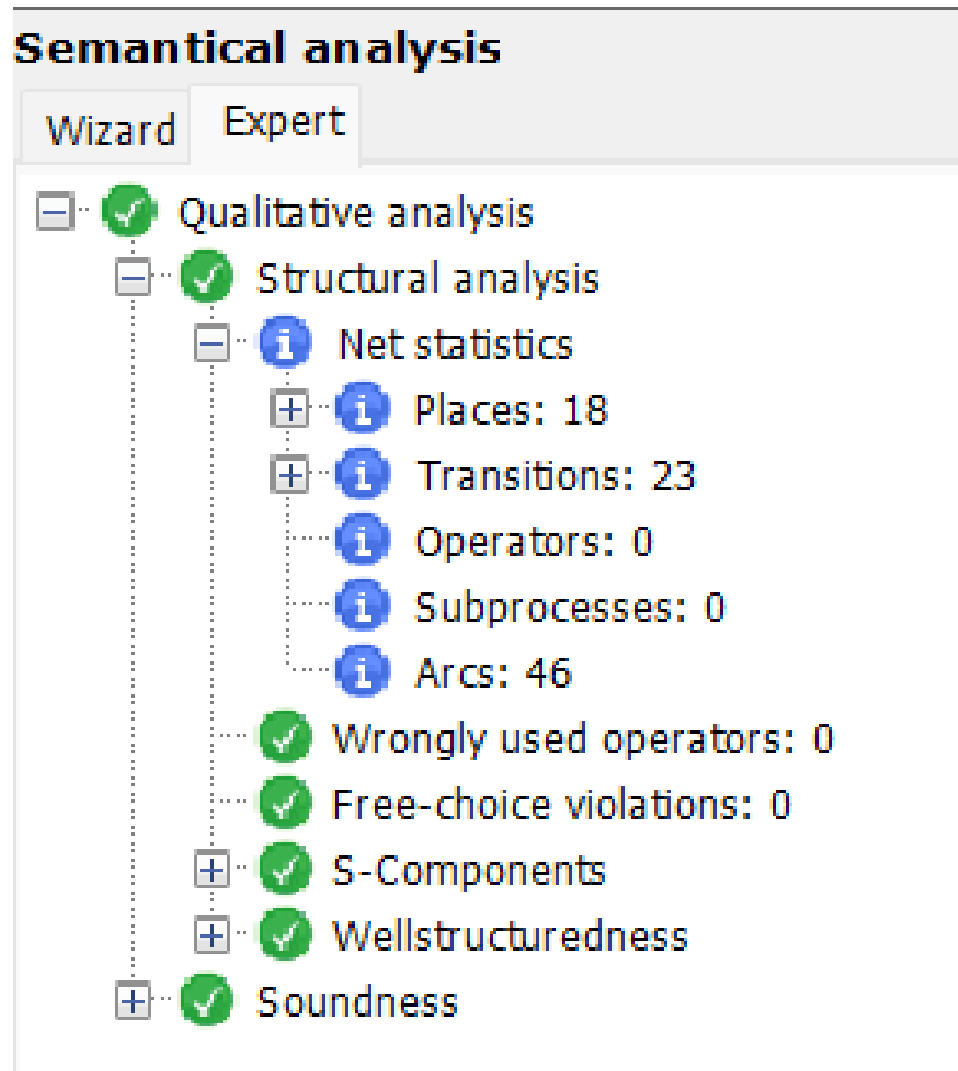


Figure 4.4: Semantical analysis in WoPeD: verified workflow net property and soundness; 18 places, 23 transitions, 46 arcs.

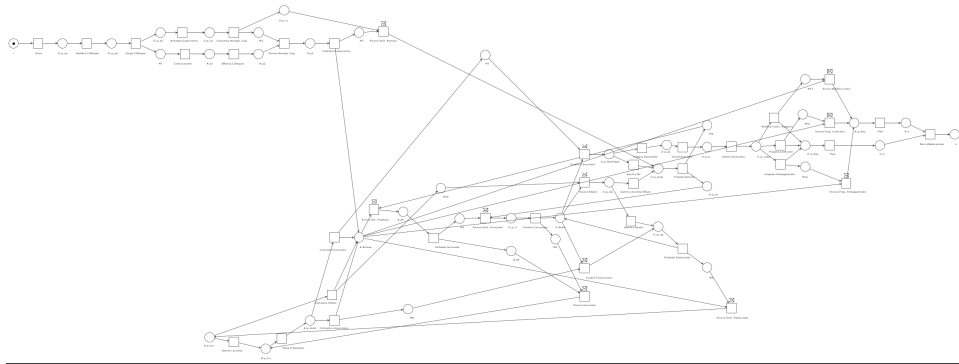


Figure 4.5: Collaboration's workflow net: the two orchestrations combined via 13 place-messages (circle with an envelope.)

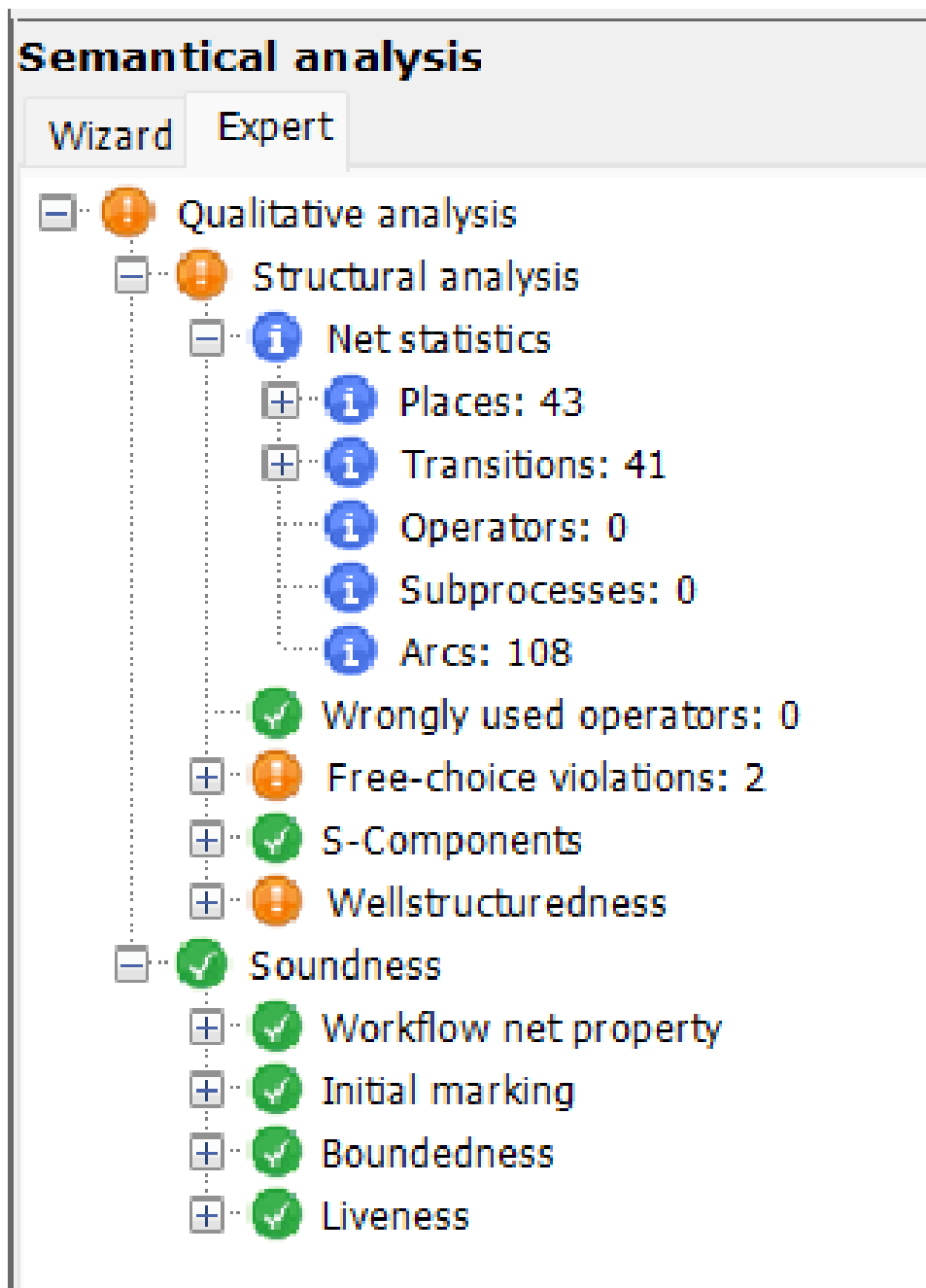
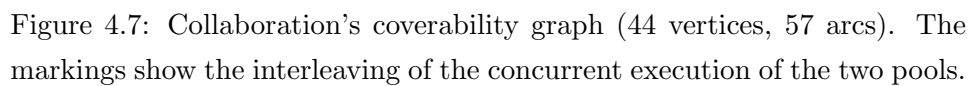


Figure 4.6: Semantical analysis in WoPeD: verified workflow net properties and soundness checked on the reachability graph; the flag for soundness signals the structural properties (free-choice-violations and handle), as expected for a non-free-choice net. 43 places, 41 transitions, 108 arcs.



Chapter 5

Conclusions

The BPMN model representing the internship scenario and the corresponding workflow nets have been designed. The two orchestrations are sound, safe, free-choice, well-handled and S-covered by a single S-component. The collaboration net, obtained by connecting the two pools through places-messages, remain sound and safe, but not free-choice (due to the two violations on the event-based gateways) nor well-handled (handle PT/TP), and it is not S-covered: this behaviour is however expected when autonomous actors cooperate through exchange of messages, consistent with what has been observed in other analogous scenarios.